



MODEL
CONTEXT
PROTOCOL



LEARN



BUILD



INTEGRATE

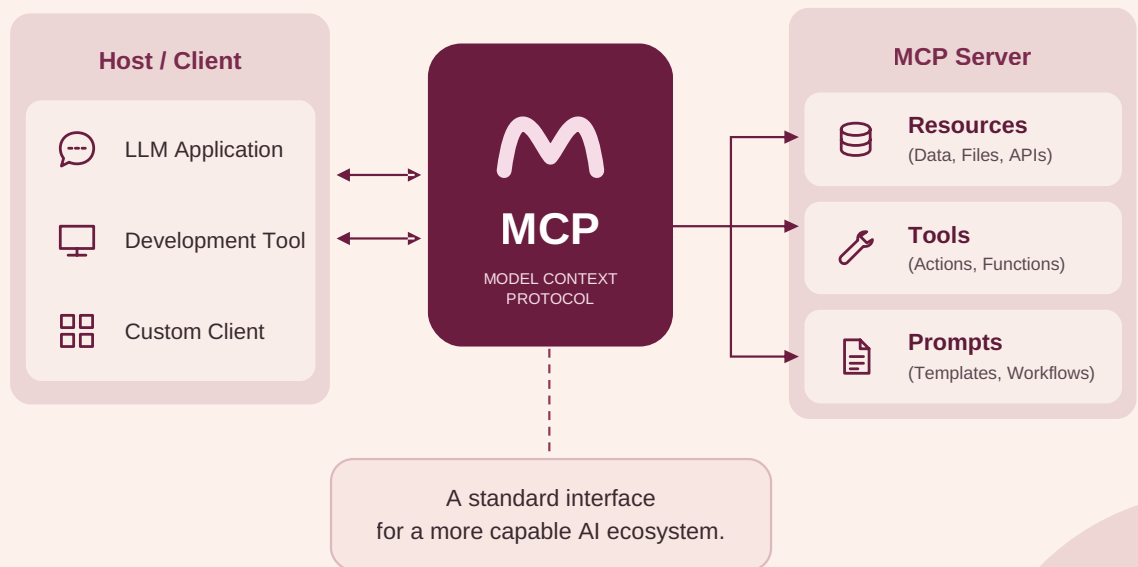


EXTEND

CHAPTER 06

Building Local MCP Servers

Build an MCP server with the SDK v2, test it in the Inspector, and connect it to Claude Desktop.



Premium Revision Notes

CONCEPTS

Clear Explanations

IMPLEMENTATION

Practical Code

DIAGRAMS

Visual Learning

INTERVIEW PREP

Key Questions

Building Local MCP Servers

Chapter 5 connected Claude Desktop to servers that other people had written. This chapter crosses to the other side of the protocol: you write the server. Using the **MCP Python SDK v2**, you will build a small demonstration server to learn the full workflow — install, write, test, connect — and then grow it, one capability at a time, into a working **Expense Tracker** that stores transactions in SQLite and lets you manage them from Claude Desktop in plain English.

Everything here is *local*: the server runs on your own machine and Claude Desktop talks to it over the **STDIO transport**. Making the same server reachable over the network is a later part of this series. All code in this chapter is written for the current SDK generation (v2, `MCPServer`); the project source files are the reference for every implementation detail.

REAL WORLD INSIGHT

Natural-language expense tracking removes the form-filling that makes most tracking apps tedious. The value is not the SQL — it is that the LLM turns “add ₹500 for a cab ride yesterday” into a structured tool call with a resolved date, an amount, and a category, and later turns “what did I spend on health last month?” into a date-ranged summary query.

1. The SDK Landscape: From Protocol to Library

Why an SDK exists

MCP is a *protocol*: a set of rules for JSON-RPC messages, lifecycle, capability negotiation, and transports. Implementing those rules by hand for every server is difficult for a beginner and highly redundant — two servers would repeat the same protocol plumbing. The official **MCP Python SDK** (the `mcp` package) removes that burden: you write ordinary Python functions and decorate them, and the SDK performs initialization, capability advertisement, request routing, schema generation, and transport handling.

Old tutorials vs. the current SDK (v2)

Many older tutorials use a class called `FastMCP`, imported from `mcp.server.fastmcp`. In SDK v2 that high-level class was renamed **`MCPServer`** and the old module path was removed, so v1-era code fails on import under `mcp 2.x`. The decorator-based programming model is otherwise the same. A separately published third-party package that also carries the “FastMCP” name is a different project and is not used in this series.

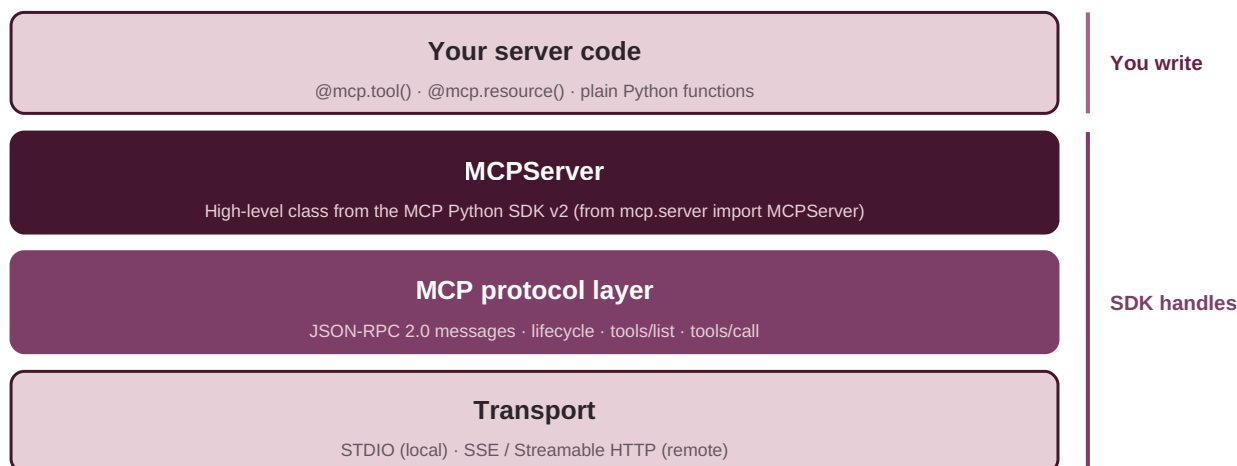


Figure 6.1 — Your functions sit on top of MCPServer; the SDK supplies the protocol layer and the transport.

Older tutorials (SDK v1)	This series (SDK v2)
<pre>from mcp.server.fastmcp import FastMCP</pre>	<pre>from mcp.server import MCPServer</pre>
<pre>FastMCP("Demo")</pre>	<pre>MCPServer("Demo")</pre>
Transport options (host, port, paths) set in the constructor	Passed to <code>run()</code> instead; the constructor keeps identity and behaviour only
Synchronous tool functions run inline on the event loop	Synchronous tool functions run on a worker thread
Protocol result fields in camelCase (<code>isError</code>)	Snake_case attributes (<code>is_error</code>); wire format unchanged
Unnamed server reports the name “FastMCP”	Unnamed server reports the name “mcp-server”

COMMON MISTAKE

Copying a v1 snippet into a v2 project. The first symptom is `ModuleNotFoundError: No module named 'mcp.server.fastmcp'`. The fix is the import and class rename shown above — the `@mcp.tool()`, `@mcp.resource()` and `mcp.run()` call sites stay the same.

🔄 Quick Revision

- MCP is the protocol; the `mcp` package is the official Python SDK that implements it.
- In SDK v2 the high-level server class is **MCPServer** (formerly `FastMCP`); import it with `from mcp.server import MCPServer`.
- Decorators `@mcp.tool()` and `@mcp.resource()` and `mcp.run()` are unchanged from v1.
- Transport settings move from the constructor to `run()`; sync handlers now run on a worker thread.

2. Project Setup with uv

uv is a fast Python package and project manager. It creates the project, the virtual environment, and the dependency list, and it can run any command inside that environment. The project used in this chapter is described by the following `pyproject.toml` (trimmed to the relevant fields):

```
[project]
name = "1-local-mcp-servers"
version = "0.1.0"
requires-python = ">=3.14"
dependencies = [
    "mcp[cli]>=2.2.0",
]
```

The `[cli]` extra installs the command-line tools used later in this chapter (`mcp dev`, `mcp run`, `mcp install`). The `>=2.2.0` pin guarantees the v2 API used by the source code.

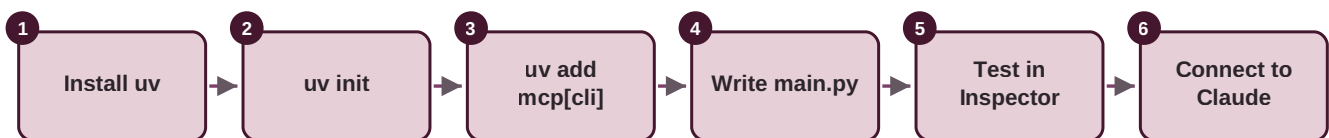


Figure 6.2 — The six-step workflow reused for every local server in this series.

Step	Command (Windows)	What it does
Install uv	<code>pip install uv</code>	Installs the package manager; <code>uv --version</code> confirms it.
Locate uv	<code>where uv</code>	Prints the absolute path of <code>uv.exe</code> — needed later for the Claude Desktop config.
Create project	<code>mkdir <folder></code> , open it in VS Code, then <code>uv init .</code>	Generates <code>pyproject.toml</code> and a starter <code>main.py</code> .
Add SDK	<code>uv add "mcp[cli]"</code>	Installs the MCP SDK and its CLI into the project environment.
Verify	<code>uv run mcp version</code>	Prints the installed SDK version (for example <code>MCP version 2.2.0</code>).

BEST PRACTICE

Always run project commands through `uv run` so they execute inside the project environment. Confirming `uv run mcp version` reports a 2.x version before you write any code prevents the v1/v2 mismatch described in Section 1.

🔄 Quick Revision

- `uv` creates the project, environment and dependency list; `uv add "mcp[cli]"` installs the SDK.
- The project pins `mcp[cli]>=2.2.0`; the `[cli]` extra provides the `mcp` command.
- `where uv` gives the absolute path that the Claude Desktop config will need.
- The reusable workflow: install → init → add SDK → write → test → connect.

3. Anatomy of an MCP Server

The first server is deliberately trivial — two tools — because the goal is to learn the process, not the logic. The skeleton is identical for every server in this chapter: import the class, create an instance, decorate functions, and call `run()`.

```
import random
from mcp.server import MCPServer

mcp = MCPServer("Demo Server")

@mcp.tool()
def roll_dice(n_dice: int = 1) -> list[int]:
    """Roll n_dice six-sided dice and return the results."""
    return [random.randint(1, 6) for _ in range(n_dice)]

@mcp.tool()
def add_numbers(a: float, b: float) -> float:
    """Add two numbers."""
    return a + b

if __name__ == "__main__":
    mcp.run()
```

Nothing in these functions mentions MCP. The decorator registers each one as a **tool**, and the SDK derives the tool definition that clients see in `tools/list` from the function itself:

Python element	Becomes in MCP
Function name (<code>add_numbers</code>)	Tool name
Docstring	Tool description — what the LLM reads to decide when to call it
Parameters and type hints	JSON Schema for the tool's input (<code>integer</code> , <code>number</code> , defaults, required fields)
Return value	Tool result content (a list returns one text block per element); scalar returns also fill structured content
<code>MCPServer("Demo Server")</code>	Server name reported to the client during initialization
<code>mcp.run()</code>	Starts the server; with no arguments it uses the STDIO transport

IMPORTANT

The description and type hints are the *only* information the LLM has about a tool. Always give every tool a docstring, and annotate parameters. For example, calling `add_numbers` with 234 and 567 returns the text `801.0` together with structured content `{"result": 801.0}`.

REAL WORLD INSIGHT

In SDK v2 a plain `def` tool runs on a worker thread, so a slow synchronous function no longer blocks other requests. If a handler must stay on the event loop, declare it `async def`.

 Quick Revision

- A server is four things: import `MCPServer`, create an instance, decorate functions, call `mcp.run()`.
- Function name, docstring, and type hints become the tool's name, description, and input schema.
- `mcp.run()` defaults to STDIO — the right transport when server and client share a machine.
- Sync tools run on a worker thread in v2; use `async def` to stay on the event loop.

4. Testing with the MCP Inspector

The **MCP Inspector** is Anthropic's debugging client for MCP servers — comparable to Postman for APIs. Testing here first is far faster than restarting Claude Desktop after every change. The SDK CLI starts your server and the Inspector together:

```
uv run mcp dev main.py
```

The Inspector opens in the browser. Confirm the transport type is **STDIO** (server and client are on the same machine), leave the rest unchanged, and click **Connect**. The tabs mirror the three MCP primitives — Resources, Prompts, Tools — plus history, ping, and the other capabilities from earlier chapters. Open **Tools** → **List Tools**, select a tool, fill in its arguments, and click **Run Tool**. The history pane shows the exact JSON-RPC traffic.

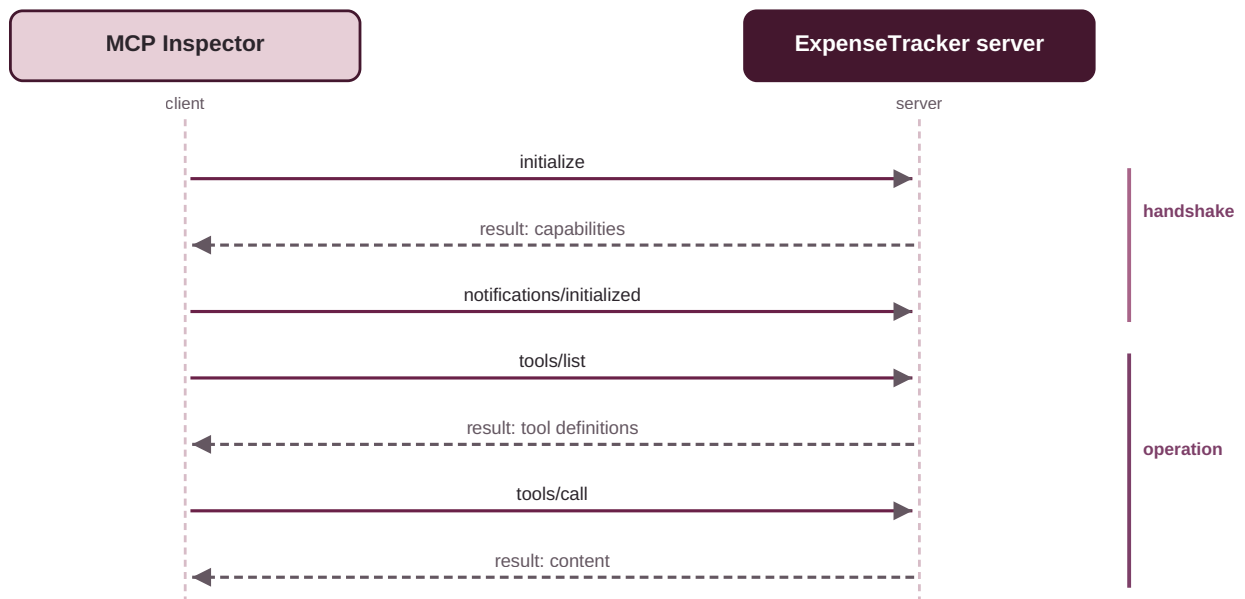


Figure 6.3 — The Inspector session: a handshake, then `tools/list` and `tools/call` in the operation phase.

BEST PRACTICE

Debug in the Inspector first, integrate with Claude Desktop second. If a tool fails in the Inspector you see the raw request and error; in Claude Desktop you only see the model’s interpretation of it.

COMMON MISTAKE

Running `mcp dev` without Node.js. The Inspector is launched through `npx`, so Node.js and npm must be installed and on the PATH; otherwise the command reports that `npx` was not found.

Quick Revision

- `uv run mcp dev main.py` launches the server and the MCP Inspector together (Node.js required).
- Select STDIO, click Connect; List Tools sends `tools/list`, Run Tool sends `tools/call`.
- The history pane shows the full JSON-RPC exchange, including the initialization handshake.
- `uv run mcp run main.py` starts the server on its own with no Inspector.

5. Connecting the Server to Claude Desktop

Claude Desktop learns about local servers from `claude_desktop_config.json` (Settings → Developer → Edit Config). Each entry under `mcpServers` gives a command and its args; at startup Claude Desktop launches that command

as a subprocess and speaks MCP with it over STDIO. This chapter uses the demonstration server and the Expense Tracker:

```
{
  "mcpServers": {
    "my-demo-server": {
      "command": "C:/Users/<USER>/local/bin/uv.exe",
      "args": [
        "run",
        "--project",
        "<PROJECT_DIR>",
        "python",
        "<PROJECT_DIR>/main.py"
      ]
    },
    "ExpenseTracker": {
      "command": "C:/Users/<USER>/local/bin/uv.exe",
      "args": [
        "run",
        "--project",
        "<PROJECT_DIR>",
        "python",
        "<PROJECT_DIR>/main.py"
      ]
    }
  }
}
```

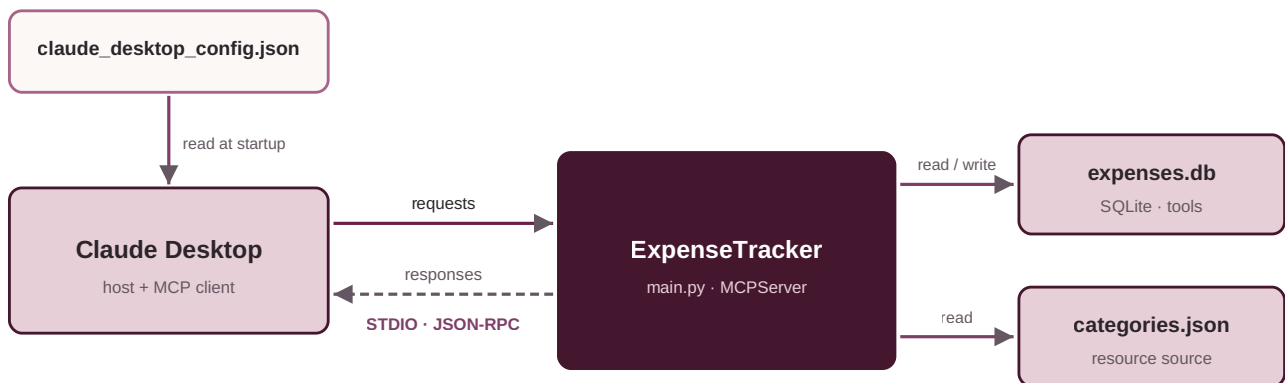


Figure 6.4 — Claude Desktop reads the config, launches the server, and exchanges JSON-RPC over STDIO.

Config part	Meaning
<code>command</code>	Executable to launch — the absolute path to <code>uv.exe</code> (from where <code>uv</code>)
<code>args: run</code>	<code>uv</code> sub-command that executes a program inside the project environment
<code>--project <PROJECT_DIR></code>	Tells <code>uv</code> which project (its <code>pyproject.toml</code> and virtual environment) to use, regardless of Claude Desktop's working directory
<code>python <PROJECT_DIR>/main.py</code>	The program to run: your server file, started with the project's Python

COMMON MISTAKE

Writing just `"uv"` as the command. Claude Desktop may not find it and reports that it cannot connect to the server. Replace it with the absolute path from where `uv`, save, then **fully quit** Claude Desktop (including the system-tray icon) and reopen it.

BEST PRACTICE

The SDK can write an entry for you: `uv run mcp install main.py` installs the server into Claude Desktop, pinning the spawned environment to your installed SDK version. The hand-written `--project` form above gives you explicit control over the command, and is the schema used throughout this series. Once `main.py` holds the Expense Tracker, the demo entry launches the same file and can be removed.

🔄 Quick Revision

- Config entries live under `mcpServers`, each with `command` and `args`; Claude Desktop spawns the process and talks over `STDIO`.
- Use the absolute path to `uv.exe` and `--project` to pin the environment.
- Config and code changes take effect only after fully quitting and reopening Claude Desktop.
- `uv run mcp install main.py` is the CLI shortcut for registering a server.

6. Building the Expense Tracker

The Expense Tracker is built incrementally: a minimal version first, then one improvement per step. After each change you save `main.py`, restart Claude Desktop, and test with natural language. The final file contains five tools and one resource.

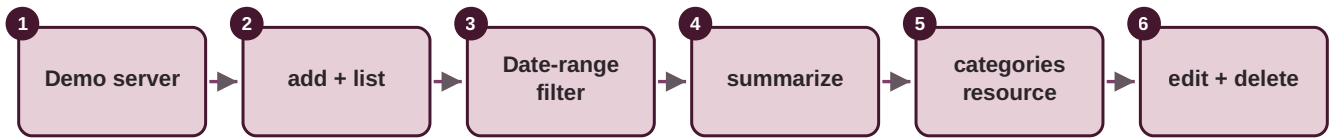


Figure 6.5 — Incremental build order; the last stage is the complete supplied project file.

Storage: SQLite

Transactions are stored in `expenses.db`, a SQLite file inside the project folder. SQLite needs no server and is the simplest option for learning; a production system would use a full database such as PostgreSQL or MySQL. The server creates the table on startup if it does not exist:

```

from mcp.server import MCPServer
import os
import sqlite3

DB_PATH = os.path.join(os.path.dirname(__file__), "expenses.db")
CATEGORIES_PATH = os.path.join(os.path.dirname(__file__), "categories.json")

mcp = MCPServer("ExpenseTracker")

def init_db():
    with sqlite3.connect(DB_PATH) as c:
        c.execute("""
            CREATE TABLE IF NOT EXISTS expenses(
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                date TEXT NOT NULL,
                amount REAL NOT NULL,
                category TEXT NOT NULL,
                subcategory TEXT DEFAULT '',
                note TEXT DEFAULT ''
            )
        """)

init_db()

```

Column	Type	Purpose
<code>id</code>	INTEGER, auto-increment	Unique key for every transaction
<code>date</code>	TEXT, required	Transaction date, stored as an ISO YYYY-MM-DD string
<code>amount</code>	REAL, required	Amount of the transaction
<code>category / subcategory</code>	TEXT (subcategory defaults to empty)	Classification, e.g. transport / cab_ride_hailing
<code>note</code>	TEXT, default empty	Free-text detail, e.g. “Cab to airport”

Paths are built from `__file__`, so the database and category file are always found next to `main.py` — even though Claude Desktop launches the process from elsewhere.

Tool 1 — `add_expense`

Claude supplies the arguments; the tool inserts one row and returns its new id. Relative dates such as “yesterday” are resolved by the LLM into a concrete date *before* the tool is called.

```
@mcp.tool()
def add_expense(date, amount, category, subcategory = "", note = ""):
    '''Add a new expense entry to the database.'''
    with sqlite3.connect(DB_PATH) as c:
        cur = c.execute(
            "INSERT INTO expenses(date, amount, category, subcategory, note) VALUES
            (?, ?, ?, ?, ?)",
            (date, amount, category, subcategory, note)
        )
        return {"status": "ok", "id": cur.lastrowid}
```

Tool 2 — `list_expenses (date range)`

The first version returned every row. Listing everything is rarely useful, so the tool takes a `start_date` and `end_date` and filters with `BETWEEN`. A request like “expenses from the first week of September” becomes a call with both dates filled in by the LLM. The dictionary-building idiom (`cur.description` supplies column names) turns each row into a JSON-friendly object.

```
@mcp.tool()
def list_expenses(start_date, end_date):
    '''List expense entries within an inclusive date range.'''
    with sqlite3.connect(DB_PATH) as c:
        cur = c.execute(
            """
            SELECT id, date, amount, category, subcategory, note
            FROM expenses
            WHERE date BETWEEN ? AND ?
            ORDER BY id ASC
            """,
            (start_date, end_date)
        )
        cols = [d[0] for d in cur.description]
        return [dict(zip(cols, r)) for r in cur.fetchall()]
```

IMPORTANT

Dates are stored as `TEXT`, and `BETWEEN` compares them as strings. That only orders correctly for ISO `YYYY-MM-DD` values, so every date passed to these tools must use that format.

Tool 3 — summarize

Questions such as “how much did I spend on education in the last ten days?” need an aggregate, not a list. `summarize` totals amounts per category in a date range, with an **optional** category filter: the base query is built first, and the extra condition and its parameter are appended only when a category is given.

```
@mcp.tool()
def summarize(start_date, end_date, category=None):
    '''Summarize expenses by category within an inclusive date range.'''
    with sqlite3.connect(DB_PATH) as c:
        query = (
            """
            SELECT category, SUM(amount) AS total_amount
            FROM expenses
            WHERE date BETWEEN ? AND ?
            """
        )
        params = [start_date, end_date]

        if category:
            query += " AND category = ?"
            params.append(category)

        query += " GROUP BY category ORDER BY category ASC"

        cur = c.execute(query, params)
        cols = [d[0] for d in cur.description]
        return [dict(zip(cols, r)) for r in cur.fetchall()]
```

Tools 4 and 5 — edit_expense and delete_expense

The complete project also lets you correct or remove a record. Both check `cur.rowcount`: if no row matched the id, they return an explanatory error string instead of silently succeeding, which lets the LLM tell you what went wrong.

```

@mcp.tool()
def edit_expense(id, date, amount, category, subcategory = "", note = ""):
    '''Edit any attribute in a particular expense.'''
    with sqlite3.connect(DB_PATH) as c:
        cur = c.execute(
            """
            UPDATE expenses
            SET date = ?, amount = ?, category = ?, subcategory = ?, note = ?
            WHERE id = ?
            """,
            (date, amount, category, subcategory, note, id)
        )

        if cur.rowcount == 0:
            return f"Error : Failed to edit the entry. No expense record found with id : {id}."

        return f"Success: Expense entry ID : {id} has been updated to amount {amount} under '{category}'."

@mcp.tool()
def delete_expense(id):
    '''Delete any particular expense by it's id.'''
    with sqlite3.connect(DB_PATH) as c:
        cur = c.execute(
            """
            DELETE FROM expenses
            WHERE id = ?
            """,
            (id,)
        )

        if cur.rowcount == 0:
            return f"Error: Failed to delete entry. No expense record found with ID: {id}."

        return f"Success: Expense entry ID {id} has been permanently deleted from the database."

```

BEST PRACTICE

Every query uses ? placeholders with a separate parameter tuple, never string formatting of user values. Even in summarize, only fixed SQL fragments are concatenated; the values stay in params. This keeps LLM-supplied text from altering the SQL.

REAL WORLD INSIGHT

Each tool opens its own `sqlite3.connect()` inside the function body. Because v2 runs sync tools on worker threads, this per-call connection pattern means no connection object is ever shared between threads.

COMMON MISTAKE

Leaving parameters un-annotated. In the supplied `main.py` the tool parameters have no type hints, so the schema returned by `tools/list` types every argument — including `amount` and `id` — as `string` (checked against SDK 2.2.0). It works, but hints such as `amount: float` and `id: int` give clients an accurate schema.

🔄 Quick Revision

- The Expense Tracker persists to a SQLite file, `expenses.db`, created on startup by `init_db()`.
- Five tools: `add_expense`, `list_expenses`, `summarize`, `edit_expense`, `delete_expense`.
- Dates are ISO strings; the LLM converts phrases like “last week” into `start_date/end_date`.
- `summarize` appends its optional category filter only when one is supplied.
- Parameterized queries and `rowcount` checks make the tools safe and self-explanatory.

7. Resources: Enforcing Consistent Categories

The problem

When Claude adds an expense it invents the category itself. Today it might write *Education*, tomorrow *upskilling* or *education* in lower case — and it ignores the subcategory column entirely. The sample database shows exactly this drift: it holds both *Entertainment* and *entertainment*, and a separate *Groceries* category next to *Food*. Inconsistent labels split totals and break later analysis.

The solution

Give the model a fixed vocabulary. `categories.json` maps each of 20 categories to its list of subcategories (119 in total); for example “transport” contains `fuel`, `cab_ride_hailing`, `parking`, and so on. The server exposes this file as an MCP **resource**:

```
@mcp.resource("expense://categories", mime_type="application/json")
def categories():
    # Read fresh each time so you can edit the file without restarting
    with open(CATEGORIES_PATH, "r", encoding="utf-8") as f:
        return f.read()

if __name__ == "__main__":
    mcp.run()
```

The decorator takes a **URI** that identifies the resource (`expense://categories`) and a MIME type; the function simply returns the file’s text. Because it reads the file on every request, you can add categories without restarting the server. In Claude Desktop the resource is attached from the + menu (*Add from ExpenseTracker* → *categories*); its JSON then sits in the conversation. When you add “cab ride to the airport, ₹700”, Claude now selects *transport* and a matching subcategory from the list instead of improvising.

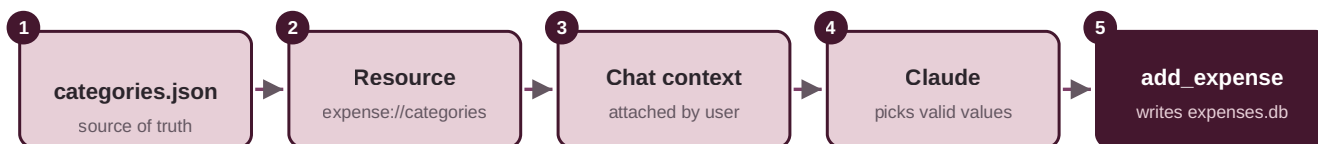


Figure 6.6 — A resource supplies the vocabulary; the tool call then writes only valid categories.

	Tool	Resource
Purpose	Performs an action (insert, update, query)	Provides data as context (the category list)
Defined with	<code>@mcp.tool()</code>	<code>@mcp.resource(uri)</code>
Identified by	Name and input schema	URI such as <code>expense://categories</code>
Used in this server	Add, list, summarize, edit, delete	Constrain category and subcategory values

INTERVIEW TIP

Be ready to explain when to choose a resource over a tool: a resource exposes read-only context the host can attach, while a tool lets the model perform actions. Here the resource does not change the database — it narrows what the model is allowed to write.

REAL WORLD INSIGHT

The same file drives consistency across sessions: because categories live in one JSON document, adding a new category is a data edit, not a code change.

🔄 Quick Revision

- A resource is exposed with `@mcp.resource(uri, mime_type=...)`; the function returns the content.
- `expense://categories` serves `categories.json` — 20 categories, 119 subcategories.
- Attaching the resource in Claude Desktop keeps category and subcategory values consistent.
- Reading the file per request lets you edit categories without restarting the server.

8. Running, Verifying, and Troubleshooting

After each code change: save `main.py`, fully quit and reopen Claude Desktop, and confirm the tool list under the ExpenseTracker entry. Then try prompts such as “Add ₹500 for a cab ride yesterday”, “Show all expenses from September in a table”, and “How much did I spend on health last month?”. Claude asks permission for a tool call (the permission flow from Chapter 5); choosing to always allow is reasonable for a harmless personal server. Open `expenses.db` in a SQLite viewer in VS Code to confirm rows are being written.

Symptom	Likely cause	Fix
Server missing or cannot connect in Claude Desktop	Bare <code>uv</code> command not resolvable	Use the absolute path from <i>where</i> <code>uv</code> ; fully restart Claude
New or changed tool not visible	Claude Desktop launches the server at startup	Save, quit completely, reopen
No module named <code>'mcp.server.fastmcp'</code>	v1 import with SDK 2.x	<code>from mcp.server import MCPServer</code>
<code>MCPServer()</code> got an unexpected keyword <code>'port'</code>	Transport settings moved	Pass them to <code>run()</code>
<code>mcp dev</code> reports <code>npx</code> not found	Node.js missing	Install Node.js and reopen the terminal
Inconsistent categories in the database	Model invents category names	Attach the <code>expense://categories</code> resource

🔄 Quick Revision

- Loop: edit → save → restart Claude Desktop → test in natural language → inspect `expenses.db`.
- Most connection failures are path problems: absolute `uv.exe`, correct `--project` folder.
- Old-SDK errors (missing `fastmcp` module, constructor keywords) map directly to the v2 changes in Section 1.
- Use the Inspector for raw protocol-level debugging before blaming Claude.

Key Takeaways

- SDK v2 replaces **FastMCP** with **MCPServer**: `from mcp.server import MCPServer`, then `@mcp.tool()`, `@mcp.resource()` and `mcp.run()`.
- Function names, docstrings, and type hints **are** the tool contract the LLM sees.
- Workflow: `uv project` → SDK install → write server → test in the Inspector → register in Claude Desktop.
- Claude Desktop launches each local server as a subprocess and speaks JSON-RPC over STDIO; use absolute paths and restart fully after every change.
- The Expense Tracker = SQLite + five tools + one resource; parameterized SQL and ISO dates keep it safe and predictable.
- A resource (`expense://categories`) constrains category and subcategory values, keeping stored data consistent.

Revision Sheet

Five compact cards summarise the commands, code shapes, and configuration to recall without rereading the chapter.

Commands

- `where uv` — absolute path for the config
- `uv init . · uv add "mcp[cli]" · uv run mcp version`
- `uv run mcp dev main.py` — Inspector
- `uv run mcp run main.py · uv run mcp install main.py`

Server skeleton

Import `MCPServer` → `mcp = MCPServer("Name")` → decorate functions with `@mcp.tool()` or `@mcp.resource(uri)` → `mcp.run()` under `if __name__ == "__main__":`.

Claude Desktop entry

`mcpServers` → server key → `command` (absolute `uv.exe`) + args (`run, --project <dir>, python, <dir>/main.py`). Quit and reopen Claude after edits.

Expense Tracker tools

`add_expense`, `list_expenses(start_date, end_date)`, `summarize(start_date, end_date, category=None)`, `edit_expense`, `delete_expense` — all backed by one `expenses` table in SQLite.

Resource

`@mcp.resource("expense://categories", mime_type="application/json")` returns `categories.json`: 20 categories, 119 subcategories, re-read on every request.

Interview Questions

Six questions that test whether the concepts, not just the commands, have stuck.

Q1. What changed between the v1 FastMCP class and SDK v2? The class is now `MCPServer`, imported from `mcp.server` (module `mcp.server.mcpserver`); the old `mcp.server.fastmcp` path was removed, transport settings moved to `run()`, and sync handlers run on worker threads. The decorators are unchanged.

Q2. How does the SDK turn a Python function into an MCP tool? The decorator registers it; the name, docstring, and type-annotated parameters become the tool name, description, and JSON input schema, and the return value becomes result content.

Q3. Why test with the MCP Inspector before Claude Desktop? It shows raw JSON-RPC requests and results for `tools/list` and `tools/call`, isolating server bugs from model behaviour, and avoids restarting the host after each change.

Q4. Why does the Claude Desktop config use the absolute path to `uv.exe` and `--project`? The host may not resolve a bare `uv`, and `--project` pins the correct project environment regardless of the launch directory.

Q5. Why expose categories as a resource instead of a tool? It is read-only context that constrains what the model writes; it prevents drift such as *Education* vs *education* without adding an action.

Q6. Why use `?` placeholders in the SQL? Values supplied by an LLM travel separately from the query text, so they cannot change the SQL structure.

Further Reading

- MCP Python SDK repository — github.com/modelcontextprotocol/python-sdk
- SDK v1 → v2 migration guide — github.com/modelcontextprotocol/python-sdk/blob/main/docs/migration.md
- Model Context Protocol documentation — modelcontextprotocol.io
- The `mcp` package on PyPI — pypi.org/project/mcp